UNIVERSITY: _____

FACULTY / DEPARTMENT: _____

FINAL PROJECT REPORT

Vietnamese Social Media Lexical Normalization with BARTpho

A Comparative Study of Gold and LLM-Reviewed Weak-Label Training

Course: Statistical Learning

Instructor: _____

Student 1: _____ **ID:** _____

Student 2: _____ **ID:** _____

Student 3: _____ **ID:** _____

Student 4: _____ **ID:** _____

Submission date: _____

Abstract

Vietnamese social media text frequently contains abbreviations, informal spellings, omitted diacritics, and other non-standard lexical forms. This study formulates their normalization as a sequence-to-sequence task and compares three BARTpho-syllable configurations. Model A was fine-tuned on 8,372 human-annotated ViLexNorm training pairs. Model B was initialized from Model A and trained with a balanced mixture of gold data and an initial pool of 18,970 LLM-reviewed weak labels. Model C was also initialized from Model A but used an expanded pool of 64,813 LLM-reviewed weak labels. Candidate normalizations for ViSoLex were reviewed by Gemini under a KEEP, EDIT, or REJECT protocol and then subjected to deterministic validation. A common Dev evaluation selected Model C before final Test scoring. On the 1,045-sentence ViLexNorm Test split, Models A, B, and C obtained F1 scores of 0.7182, 0.7422, and 0.7650, and Error Reduction Rates of 0.6002, 0.6331, and 0.6647, respectively. Model C was used for a local Gradio application, with Model B retained as a fallback checkpoint. The application performs BARTpho inference locally and does not use the review model at runtime.

1 Introduction

Vietnamese social media messages often depart from conventional written forms. Users may shorten words, omit diacritics, write phonetic variants, or use informal lexical forms that are understandable in context but difficult for downstream language-processing systems to interpret consistently. For example, the sentence *mik ko bt hnay đi hc ko* can be normalized as *mình không biết hôm nay đi học không*. This transformation changes lexical forms such as *mik*, *ko*, *bt*, *hnay*, and *hc*, while preserving the intended proposition and the overall sentence structure.

The present work treats this problem as lexical normalization rather than unrestricted rewriting. The intended output is a more canonical lexical realization of the input, not a paraphrase, a stylistic reformulation, a full grammatical correction, or an alteration of sentiment and content. This distinction matters because a system that rewrites a sentence fluently may still be unsuitable for lexical normalization if it changes information that was not expressed in the source text.

The available supervised resource, ViLexNorm, provides human-annotated pairs of noisy and normalized Vietnamese social media sentences [6]. Its 8,372-example training split provides a direct basis for supervised fine-tuning, but social media language is diverse and its lexical variation is not fully represented by a moderate-sized gold corpus. In contrast, the processed ViSoLex corpus used in this project contains 68,411 unlabeled social media sentences. The central question of this study is therefore whether LLM-reviewed labels constructed from this unlabeled corpus can provide additional training signal for a BARTpho model.

The study compares three configurations of the same BARTpho-syllable architecture. Model A uses only ViLexNorm gold training data. Model B continues from Model A with an initial LLM-reviewed weak-label pool of 18,970 examples. Model C also continues from Model A but uses an expanded LLM-reviewed weak-label pool of 64,813 examples. The comparison is designed to examine how the composition and coverage of LLM-reviewed weak-label data are associated with lexical-normalization performance. The project further implements a local Gradio application using the model selected on Dev, so that inference does not require the training corpus or an external review API.

2 Background and Related Work

2.1 Lexical normalization

Lexical normalization maps non-standard lexical forms to conventional or canonical forms. The mapping may be one-to-one, as in an abbreviation expanded to one standard word, but it may also be one-to-many or many-to-one. A short form such as *đky* may become *đăng ký*, whereas other transformations may merge or reorder whitespace-delimited units. Consequently, the task is naturally expressed as conditional sequence generation rather than as a collection of independent token classifications.

ViLexNorm introduced a Vietnamese lexical-normalization corpus containing more than 10,000 human-annotated sentence pairs collected from public social media comments [6]. The source study evaluated systems with Error Reduction Rate (ERR) relative to a Leave-As-Is baseline and reported a BARTpho-syllable result of 57.74% under its evaluation setting. The project reported here uses the ViLexNorm Train, Dev, and Test splits as supplied, but it implements a deterministic token-edit scorer because the referenced corpus repository distributes data rather than an official evaluator. Its numerical results should therefore be interpreted within the project evaluation protocol rather than compared directly with values reported by another implementation.

2.2 From Transformer to BARTpho

The Transformer introduced an encoder–decoder architecture that relies on attention rather than recurrence or convolution [10]. In an encoder–decoder setting, the encoder constructs contextual representations of the input sequence, while the decoder produces output tokens autoregressively. Self-attention allows each token representation to use information from other positions in the same sequence. In its standard scaled dot-product form, attention is written as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right)V, \tag{1}$$

where Q, K, and V denote query, key, and value projections. The weighting operation determines how strongly each position uses information from other positions. Multi-head attention applies several such projections in parallel.

BART is a denoising sequence-to-sequence model. During pre-training, its input text is corrupted and the model learns to reconstruct the original text [3]. This objective does not directly train social media lexical normalization, but it provides an encoder–decoder initialization for conditional reconstruction. The generated sequence may differ in length from the source sequence, which is useful when a non-standard social media form must be expanded or contracted.

BARTpho adapts the BART sequence-to-sequence approach to Vietnamese [8]. It provides word-level and syllable-level variants and is based on the BART-large architecture. This project uses `vinai/bartpho-syllable`. The syllable-level variant is appropriate for the present preprocessing pipeline because noisy forms such as *mik*, *hnay*, and *khum* do not require a separate Vietnamese word-segmentation step before they are passed to the model. This choice does not imply that the syllable-level variant is universally preferable; it matches the lexical characteristics and input representation of the current task.

Shared architecture: BARTpho-syllable

Model A	**Model B**	**Model C**
Gold-only training	Gold + initial reviewed pool	Gold + expanded reviewed pool
8,372 gold pairs	8,372 gold pairs + 18,970 weak labels	8,372 gold pairs + 64,813 weak labels

Model A also provides the candidate-generation and initialization provenance for Models B and C.

Figure 1: Three BARTpho-syllable training configurations. The visual layout presents the configurations symmetrically, while the final sentence records their actual checkpoint provenance.

3 Task Formulation and Experimental Design

Let $x = (x_1, \ldots, x_m)$ denote a noisy social media sentence and let $y = (y_1, \ldots, y_n)$ denote its normalized form. A sequence-to-sequence model with parameters θ estimates the conditional probability

$$P(y \mid x; \theta) = \prod_{t=1}^{n} P(y_t \mid y_{<t}, x; \theta). \tag{2}$$

The output length n need not equal the input length m. This property permits the model to represent one-to-many and many-to-one lexical mappings without adding a separate alignment module.

The experimental variable is the reviewed weak-label data available during training. Model A receives only human-annotated ViLexNorm pairs. Models B and C are initialized from Model A and receive balanced mixtures of the same gold training data with reviewed ViSoLex weak labels. Model B uses the initial accepted pool, whereas Model C uses the expanded accepted pool. The three configurations share the same BARTpho-syllable architecture, tokenizer family, maximum sequence lengths, decoding configuration, and validation set.

The comparison should not be read as a perfect isolation of weak-label count. Model A begins from the public BARTpho checkpoint, whereas Models B and C continue from Model A. Models B and C also use different training durations because their weak-label pools have different sizes. The results consequently compare complete training configurations: initial checkpoint, data composition, sampling schedule, and number of epochs. This qualification is retained throughout the analysis.

4 Data Preparation and Weak-Label Construction

4.1 Gold data and unlabeled corpus

ViLexNorm was retained in its original Train, Dev, and Test partitioning. The training split contains 8,372 examples, the development split contains 1,050 examples, and the test split contains 1,045 examples. Each processed record stores an identifier, dataset name, split name, noisy input, normalized target, and the label source. Preprocessing normalizes whitespace and removes records with empty input or target fields, but it does not correct lexical noise before training. Retaining noise is necessary because the noise itself constitutes the input signal for normalization.

Table 1: ViLexNorm splits used by the project.

Split	Examples	Use
Train	8,372	Fine-tuning
Dev	1,050	Checkpoint selection and development analysis
Test	1,045	Comparative evaluation

Table 2: Canonical ViSoLex corpus after preprocessing and global deduplication.

Source	Original task	Retained
ViHSD [4]	Hate-speech classification	30,579
UIT-VSMEC [1]	Emotion recognition	6,916
ViHOS [2]	Offensive-span detection	0
ViSpamReviews [9]	Spam-review detection	19,805
UIT-ViSFD [7]	Aspect/sentiment analysis	11,111
Total		68,411

The additional unlabeled corpus follows the source composition released with ViSoLex [5]. The initial specification expected approximately 121,087 sentences; this is a planning estimate, not an observed raw count for the implemented run. The available inputs produced 68,411 canonical unique sentences after empty-record removal, protected-set filtering, and source-order deduplication. A complete numerical decomposition from 121,087 to 68,411 cannot be reconstructed from retained artifacts. The confirmed source-level effect is that ViHOS records duplicated ViHSD records loaded earlier and therefore contributed no additional unique sentence.

Leakage prevention is applied before weak-label construction and again when accepted records are assembled. Exact input matches with ViLexNorm Dev or Test are removed from ViSoLex. The Dev and Test inputs are also exported as SHA-256 fingerprints. During weak-label construction, an input whose normalized hash appears in this protected set is rejected. The processed-data validator further checks record schemas, unique identifiers, whitespace normalization, duplicate ViSoLex inputs, and residual Dev/Test overlap.

4.2 Candidate generation, review, and validation

Model A generated one candidate normalization for every ViSoLex sentence. Generation used a batch size of 16, four beams, a maximum source length of 128 tokens, and at most 128 generated tokens. The process was divided into chunks of 1,000 records so that completed chunks could be verified and resumed. Each candidate preserves its source identifier and provenance, records the generation-configuration hash, and stores a confidence value defined as the mean generated-token log probability, including the end-of-sequence token. Confidence is used to stratify review and audit samples; it is not itself a sufficient condition for accepting a weak label.

Each candidate was reviewed by Gemini 3.5 Flash-Lite using the frozen lexical-normalization review prompt (version 1), temperature 0, batches of at most 15 records, and a structured JSON response schema. Automatic function calling was disabled. A KEEP decision accepts the Model A

Table 3: Review outcomes and accepted weak labels. The final accepted count can be lower than the sum of KEEP and EDIT decisions because validation filters are applied afterwards.

Stage	Manifest	Valid	Keep	Edit	Reject	Accepted
Initial review	20,000	19,997	9,043	9,952	1,002	18,970
Expanded review	48,411	48,406	21,973	23,931	2,502	45,843
Combined pool	–	–	30,983	33,830	–	64,813

candidate as the target. An EDIT decision replaces it with a complete, minimally corrected target supplied by the LLM reviewer. A REJECT decision excludes the record because a reliable lexical target cannot be determined or because the transformation lies outside lexical normalization. The reviewer is used only in offline data preparation. It is not loaded or called by the deployed application.

Review decisions alone do not determine training membership. Accepted KEEP and EDIT records are filtered for empty targets, serialized-response artifacts, duplicate input text, protected-set overlap, implausible target length, and excessive character-level change. The accepted target-to-input length ratio must lie between 0.5 and 2.0, and the character edit ratio computed from `SequenceMatcher` must not exceed 0.8. These checks remove obvious failures and large rewrites that are inconsistent with the task scope. They do not establish that every remaining weak label is semantically perfect.

The initial accepted pool contains 52.38% EDIT and 47.62% KEEP records; the combined pool contains 52.20% EDIT and 47.80% KEEP records. Slightly more than half of the accepted targets therefore required LLM correction rather than direct candidate acceptance. The initial pool defines the additional data available to Model B, and the union of the initial and expanded pools defines the additional data available to Model C. Model A uses neither pool during supervised fine-tuning. These targets are referred to as LLM-reviewed weak labels, not human-verified annotations.

5 Model Architecture and Configurations

All configurations use the same BARTpho-syllable checkpoint family, represented by the mBART conditional-generation architecture. The checkpoint configuration contains a twelve-layer Transformer encoder and a twelve-layer Transformer decoder. The hidden representation size is 1,024, each encoder and decoder layer has 16 attention heads, and the feed-forward dimension is 4,096. The tokenizer vocabulary contains 40,030 entries. Dropout is 0.1 and the activation function is GELU. These architectural properties remain unchanged across Models A, B, and C.

5.1 Model A: Gold-only configuration

Model A was initialized from the public `vinai/bartpho-syllable` checkpoint and fine-tuned on the 8,372 ViLexNorm Train pairs. Its development set contains the same 1,050 ViLexNorm Dev examples used for the other configurations. Model A uses a learning rate of 3×10^{-5} and is trained for at most five epochs with early stopping based on development loss. The selected checkpoint is the one associated with the lowest validation loss under the Model A training procedure.

Within the comparative design, Model A represents the condition in which the model receives

Table 4: Shared BARTpho checkpoint architecture.

Component	Value
Encoder layers	12
Decoder layers	12
Hidden dimension	1,024
Attention heads per encoder/decoder layer	16
Feed-forward dimension	4,096
Vocabulary size	40,030
Maximum positional embeddings	1,024
Dropout	0.1
Activation	GELU

only human-annotated lexical-normalization targets. It also serves an operational role by generating ViSoLex candidates and supplying the checkpoint provenance for Models B and C. These additional functions do not change its status as one of the three evaluated configurations.

5.2 Model B: Initial reviewed weak-label configuration

Model B was initialized from the trained Model A checkpoint. In each epoch, it receives all 8,372 gold ViLexNorm Train records and 8,372 records sampled from the 18,970-example initial reviewed weak-label pool. The mixture therefore preserves a one-to-one gold-to-pseudo ratio for each epoch. Model B uses a learning rate of 2×10^{-5} and is trained for three epochs.

The pseudo-label sampler uses an unseen-first policy. It first selects weak-label identifiers that have not appeared in earlier epochs and only reuses previously selected identifiers when the unseen pool cannot satisfy the current quota. The full initial weak-label pool is covered across the three-epoch schedule. The Model B mixture manifest records a different seed for each epoch, beginning with 2026, and verifies the checksums of the gold, development, pseudo-label, and Model A checkpoint artifacts before training starts.

5.3 Model C: Expanded reviewed weak-label configuration

Model C was also initialized from Model A rather than from Model B. It uses the 64,813-example union of the initial and expanded reviewed weak-label pools. Each epoch again contains 8,372 gold records and 8,372 weak-label records. The number of epochs is determined by the weak-label coverage rule

$$\left\lceil \frac{64{,}813}{8{,}372} \right\rceil = 8. \tag{3}$$

Model C uses a learning rate of 2×10^{-5} and the same gold-to-pseudo ratio as Model B.

Across the eight-epoch Model C schedule, 62,650 weak labels are used once and 2,163 are used twice because the final epoch wraps through the pool after all examples have been exposed. Model C is trained with the designated ViLexNorm Train and Dev artifacts together with the expanded weak-label pool; the Test split is reserved for the common final evaluation.

Table 5: Training configurations for the three models. Effective batch size equals the per-device batch size multiplied by gradient accumulation steps.

Property	Model A	Model B	Model C
Initial checkpoint	Public BARTpho	Model A	Model A
Gold examples	8,372	8,372 per epoch	8,372 per epoch
Weak-label pool	0	18,970	64,813
Weak labels per epoch	0	8,372	8,372
Epochs	Up to 5	3	8
Learning rate	3×10^{-5}	2×10^{-5}	2×10^{-5}
Per-device batch size	8	8	8
Gradient accumulation	2	2	2
Effective batch size	16	16	16
Checkpoint criterion	Minimum Dev loss	Minimum Dev loss	Minimum Dev loss

6 Training Procedure

The source and target sequences are tokenized with the BARTpho tokenizer and truncated to 128 tokens. A sequence-to-sequence data collator performs dynamic padding within each batch and prepares target labels for token-level cross-entropy loss. The shared training settings use weight decay of 0.01, a warmup ratio of 0.1, gradient accumulation over two mini-batches, and seed 2026. On CUDA-enabled training runs, mixed precision is enabled. Model A is trained through Hugging Face `Seq2SeqTrainer`; Models B and C use a custom PyTorch loop because their epoch-specific mixture manifests prescribe the exact gold and pseudo-label membership of each epoch.

Despite this implementation difference, all configurations use the same principle for model selection. Development loss is computed on the ViLexNorm Dev set after each epoch or evaluation stage, and the checkpoint with the minimum observed development loss is retained. Generated development predictions use beam search with four beams and a maximum output length of 128. The training environment recorded for the released artifacts uses Python 3.12.13, PyTorch 2.10.0 with CUDA 12.8, and Transformers 5.0.0 on Kaggle GPU infrastructure; the Model A artifact records a Tesla T4 device.

Before full mixture training, Models B and C undergo a smoke procedure with 200 gold and 200 pseudo examples. The smoke run verifies that the loss can be computed, generation is non-empty, the checkpoint can be reloaded, and the Test split has not been loaded. These checks establish pipeline integrity rather than a scientific estimate of model quality.

7 Evaluation Protocol and Metrics

The common generation configuration uses seed 2026, a batch size of 16, a maximum source length of 128, at most 128 new tokens, four beams, and early stopping. Each prediction record stores its identifier, source input, target text, prediction text, model name, checkpoint checksum, and generation-configuration hash. The evaluation code rejects a prediction file if its record count, order, source text, target text, checkpoint identity, or generation configuration differs from the ex-

pected evaluation inputs.

The project scorer extracts lexical edits by splitting the source, target, and prediction strings on whitespace and aligning the token sequences with `SequenceMatcher`. Every non-equal alignment span is represented as an edit from a source-token tuple to a target-token tuple. This representation preserves one-to-many and many-to-one operations. Let G be the number of gold edits, S the number of predicted edits, and C the number of predicted edits that exactly match gold edits. Micro-averaged precision, recall, and F1 are defined as

$$\text{Precision} = \frac{C}{S}, \qquad \text{Recall} = \frac{C}{G}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4}$$

The project reports Error Reduction Rate (ERR) relative to a Leave-As-Is (LAI) baseline. Let E_{LAI} be the aggregate whitespace-token Levenshtein distance from input to reference and E_{system} the corresponding distance from model prediction to reference. The published token-accuracy reduction formula is equivalently computed as

$$ERR = \frac{E_{LAI} - E_{system}}{E_{LAI}}. \tag{5}$$

ERR equals one for perfect normalization, zero for Leave-As-Is behavior, and may become negative when a system introduces more token errors than the baseline. Unlike edit recall, it penalizes unnecessary or incorrect output changes through E_{system}. Exact sentence match is additionally reported as the proportion of sentences whose generated output equals the reference target exactly.

All three configurations are first scored on the same 1,050-example Dev split with one common metric implementation. Model C is selected because it obtains the highest Dev ERR (0.670380), edit F1 (0.759233), and exact sentence match (0.561905); the selection artifact explicitly records that Test metrics are not used. The three selected checkpoints are then evaluated on the same 1,045-example ViLexNorm Test split. The tokenizer contract, generation settings, prediction schema, and scoring procedure are held constant across Models A, B, and C. Bootstrap intervals for pairwise metric differences use 2,000 resamples and seed 2026.

8 Results and Discussion

8.1 Development-set results

The development results show a monotonic reduction in loss from Model A to Model C. Model B reduced development loss from 0.225680 to 0.205141 and increased exact sentence match from 0.520952 to 0.557143. Model C further reduced development loss to 0.193742 and reached an exact-match value of 0.561905. The magnitude of the exact-match increase from Model B to Model C is smaller than the increase from Model A to Model B, even though the loss reduction remains observable.

This pattern is consistent with the view that reviewed social-media examples add useful lexical variation beyond the gold training split. It does not establish weak-label pool size as the sole cause of improvement. Model A differs from Models B and C in initialization, while Models B and C differ in both weak-label coverage and training duration. The development evidence should therefore be read as a comparison of complete configurations rather than a single-variable ablation.

Table 6: Development-set results used for checkpoint selection and exploratory comparison.

Model	Best Dev loss	Exact sentence match
Model A	0.225680	0.520952
Model B	0.205141	0.557143
Model C	0.193742	0.561905

Table 7: Common A/B/C results on 1,045 ViLexNorm Test sentences.

Model	Gold edits	Predicted edits	Correct edits	ERR	Precision	F1	Exact match
Model A	1,778	1,703	1,250	0.600250	0.733999	0.718184	0.503349
Model B	1,778	1,690	1,287	0.633111	0.761538	0.742215	0.529187
Model C	1,778	1,707	1,333	0.664725	0.780902	0.764993	0.566507

8.2 Common benchmark comparison

Table 7 presents the three models under the same 1,045-sentence benchmark and generation configuration. Model A produced 1,250 correct edits from 1,703 predicted edits. Model B produced 1,287 correct edits from 1,690 predicted edits, increasing both precision and recall. Model C produced 1,333 correct edits from 1,707 predicted edits and obtained the highest value for every reported edit-based metric as well as the highest exact sentence match.

Relative to Model A, Model B increased F1 by 0.024030. Relative to Model B, Model C increased F1 by 0.022778 and increased ERR by 0.031614. For the Model C–Model B comparison, the bootstrap 95% interval for the F1 difference is $[0.012063, 0.033228]$, while the corresponding ERR interval is $[0.015893, 0.046803]$. These intervals are positive under the stated resampling procedure and support the higher performance of Model C in the common evaluation.

The common Test results show a progressive increase in performance across the three configurations. Model B improves on the gold-only configuration in precision, recall, ERR, F1, and exact sentence match. Model C records the highest value for each reported metric, indicating that the expanded reviewed weak-label configuration is the strongest of the three evaluated settings. The application therefore uses Model C as its default checkpoint.

8.3 Error Analysis

Each Test prediction was assigned one deterministic primary category. These labels are heuristic outcomes rather than causal explanations: in particular, one-to-many and many-to-one are determined from source/reference lengths, and zero sentence-level over-normalization labels do not imply zero false-positive edits.

Paired exact outcomes show that Model C was correct while Model A was wrong on 91 sentences, whereas the reverse occurred on 25. Against Model B, the corresponding counts were 57 and 18. Four compact examples illustrate both improvements and remaining failures.

The aggregate and qualitative results indicate that additional training data improved several ambiguous abbreviations, but errors remain for omitted diacritics, short forms such as *AE*, and preservation of every source token. The examples are illustrations rather than substitutes for ag-

Table 8: Primary error-analysis categories, shown as count (percentage) over 1,045 Test sentences.

Category	Model A	Model B	Model C
Correct	526 (50.3%)	553 (52.9%)	592 (56.7%)
Wrong	350 (33.5%)	327 (31.3%)	302 (28.9%)
Missed	54 (5.2%)	64 (6.1%)	59 (5.6%)
One-to-many	113 (10.8%)	99 (9.5%)	90 (8.6%)
Many-to-one	2 (0.2%)	2 (0.2%)	2 (0.2%)

Table 9: Selected qualitative examples from the deterministic error-analysis artifact.

Case	Text
A wrong; B/C correct	Input: *giống khẩu phần hồi covid của t ha m ha*. Reference/B/C: *... của tao ha mày ha*. A: *... của tôi ha mày ha*.
C only correct	Input: *nhìu khi teo tưởng con mèo....* Reference/C: *nhiều khi tao tưởng....* A/B use *tôi*.
All missed	Input: *AE mà sao kỳ vậy.* Reference: *Anh em mà sao kỳ vậy.* All models keep *AE*.
C regression	Input: *Nhìn nóa lại nghĩ đến con ngỗng nhà toy*. A/B normalize both *nóa* and *toy*; C leaves *nóa*.

gregate evaluation.

9 Local Web Application

The application is implemented with Gradio and runs on the local loopback address `127.0.0.1`. The deployed default checkpoint is Model C because it achieved the highest values in the common Dev comparison. Model B is retained as an available fallback checkpoint. Test results are reported only after this selection.

At runtime, the application validates the user input, tokenizes it with the local BARTpho tokenizer, performs CPU generation with four beams and maximum source and output lengths of 128, and displays the normalized text. The runtime is cached after the first load. Gemini is used only in offline weak-label construction; the web application neither imports nor calls the review provider. The model can therefore run after the checkpoint and inference dependencies have been installed locally, including under `HF_HUB_OFFLINE=1` and `TRANSFORMERS_OFFLINE=1`.

The recorded local smoke test normalized *mik ko bt hnay đi hc ko* to *mình không biết hôm nay đi học không*. The first request, including model loading and generation, took 19.888 seconds. A second request using the cached runtime took 2.286 seconds. A clean offline check using the released checkpoint recorded 15.989 seconds for model loading and inference. These values demonstrate functional local execution on the tested machine; they are not a general multi-user latency benchmark.

The application is started with `python -m visolexnorm.app.web --port 7860` and opened at

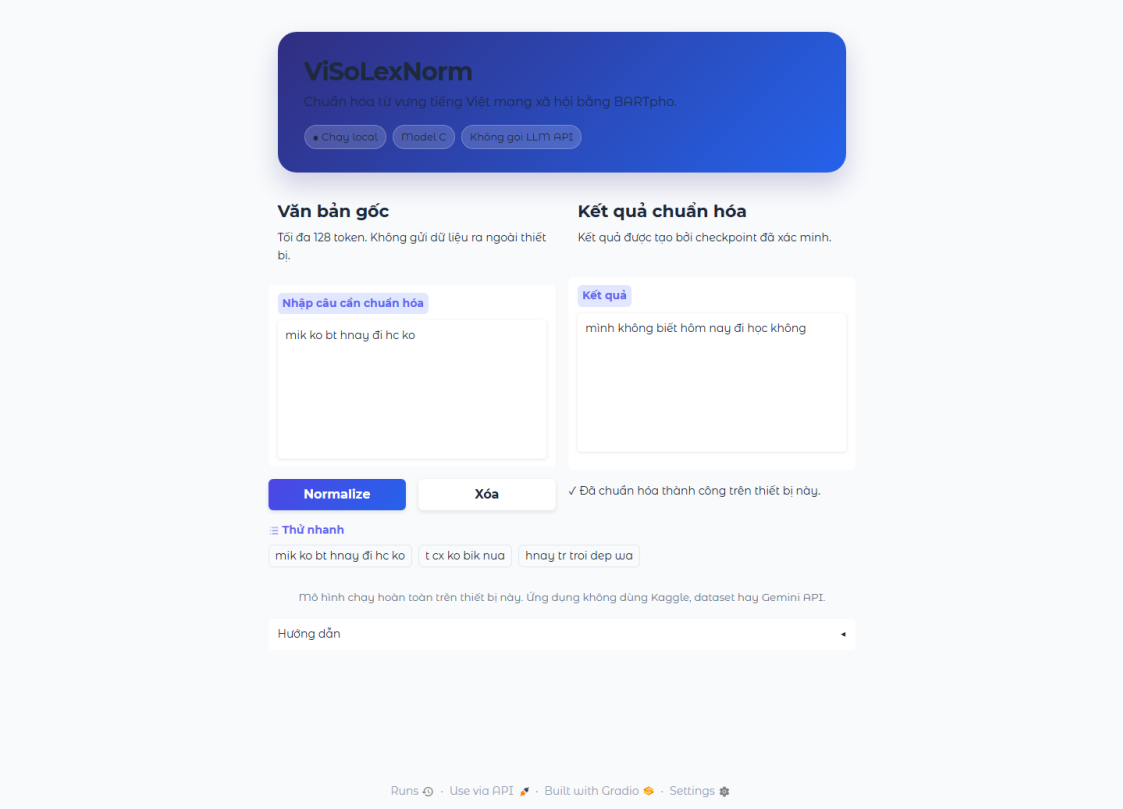

Figure 2: Local Gradio application using the Dev-selected Model C checkpoint. The displayed normalization runs locally without a Gemini API call.

`http://127.0.0.1:7860`. Figure 2 records the verified local input/output example.

10 Limitations

The LLM-reviewed weak labels are not equivalent to human annotations and were not human-verified at full scale. Temperature 0, a fixed prompt, structured schema validation, and deterministic filters improve reproducibility and reduce identifiable problems, but they do not guarantee that every accepted target is semantically optimal. Some lexical variants are context-dependent, and the LLM reviewer may select a plausible target that differs from the preferred annotation in ViLexNorm-style evaluation.

The comparison also changes more than one factor at a time. Model A begins from the public BARTpho checkpoint, whereas Models B and C begin from Model A. Model B and Model C further differ in the size of their weak-label pools and in their number of epochs. The observed differences should therefore be described as properties of the three complete training configurations, not as a causal estimate of weak-label count alone.

The edit-based metric implementation is a deterministic project scorer rather than an independently validated official ViLexNorm evaluator. In addition, the 128-token truncation limit and CPU model-loading time may affect long inputs and interactive deployment settings.

11 Conclusion

This study compared three BARTpho-syllable configurations for Vietnamese social media lexical normalization. Model A used only human-annotated ViLexNorm data, Model B added an initial pool of 18,970 LLM-reviewed weak labels, and Model C used an expanded pool of 64,813 LLM-reviewed weak labels. The common Test evaluation produced F1 values of 0.718184, 0.742215, and 0.764993 and ERR values of 0.600250, 0.633111, and 0.664725 for Models A, B, and C, respectively. The results are consistent with LLM-reviewed social-media examples providing useful additional training variation.

Model C was selected using the common Dev evaluation and is used as the default local application checkpoint, while Model B remains available as a fallback. It subsequently achieved the highest values in the common Test evaluation. Future work should expand human auditing of accepted weak labels and separate the effects of training duration, initialization, and weak-label coverage more directly.

References

[1] Vong Anh Ho, Duong Huynh-Cong Nguyen, Danh Hoang Nguyen, Linh Thi-Van Pham, Duc-Vu Nguyen, Kiet Van Nguyen, and Ngan Luu-Thuy Nguyen. Emotion recognition for vietnamese social media text. In *Computational Linguistics*, pages 319–333. Springer, 2020. doi: 10.1007/978-981-15-6168-9_27.

[2] Phu Gia Hoang, Canh Duc Luu, Khanh Quoc Tran, Kiet Van Nguyen, and Ngan Luu-Thuy Nguyen. ViHOS: Hate speech spans detection for vietnamese. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 652–669. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.eacl-main.47.

[3] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.703. URL https://aclanthology.org/2020.acl-main.703/.

[4] Son T. Luu, Kiet Van Nguyen, and Ngan Luu-Thuy Nguyen. A large-scale dataset for hate speech detection on vietnamese social media texts. In *Advances and Trends in Artificial Intelligence: Artificial Intelligence Practices*, pages 415–426. Springer, 2021. doi: 10.1007/978-3-030-79457-6_35.

[5] Anh Thi-Hoang Nguyen, Dung Ha Nguyen, and Kiet Van Nguyen. ViSoLex: An open-source repository for vietnamese social media lexical normalization. *arXiv preprint arXiv:2501.07020*, 2025. URL https://arxiv.org/abs/2501.07020. Presented at COLING 2025.

[6] Thanh-Nhi Nguyen, Thanh-Phong Le, and Kiet Nguyen. ViLexNorm: A lexical normalization corpus for Vietnamese social media text. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1421–1437, St. Julian's, Malta, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.eacl-long.85. URL https://aclanthology.org/2024.eacl-long.85/.

[7] Luong Luc Phan, Phuc Huynh Pham, Kim Thi-Thanh Nguyen, Sieu Khai Huynh, Tham Thi Nguyen, Luan Thanh Nguyen, Tin Van Huynh, and Kiet Van Nguyen. SA2SL: From aspect-based sentiment analysis to social listening system for business intelligence. In *Knowledge Science, Engineering and Management*, pages 647–658. Springer, 2021. doi: 10.1007/978-3-030-82147-0_53.

[8] Nguyen Luong Tran, Duong Minh Le, and Dat Quoc Nguyen. BARTpho: Pre-trained sequence-to-sequence models for vietnamese. In *Proceedings of the 23rd Annual Conference of the International Speech Communication Association*, 2022. URL https://arxiv.org/abs/2109.09701. Preprint: arXiv:2109.09701.

[9] Co Van Dinh, Son T. Luu, and Anh Gia-Tuan Nguyen. Detecting spam reviews on vietnamese e-commerce websites. In *Intelligent Information and Database Systems*, pages 595–607. Springer, 2022. doi: 10.1007/978-3-031-21743-2_48.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.